Tilbe Yilmaz

October 23, 2026

Dr. Tong Zhang

ECSE 4320

Project 2: Cache and Memory Performance Profiling

Contents

Introduction 2
System Configuration 2
Run Procedure 2
Code Structure and Execution Flow 2
Hypothesis 3
Experimental Controls ..... **Error! Bookmark not defined.**
Experiments and Results 3
Zero Queue Latency and Working Set 3
Pattern Granularity and Read Write Mix 4
Access Pattern and Dependent Latency 5
Latency under Different Traffic Mixes ..... **Error! Bookmark not defined.**
Throughput Latency Knee 6
Concurrency and Queueing 6
Kernel Runtime and Cache Miss Impact 7
Page Locality and Huge Pages 8
TLB Misses and AMAT Interpretation 9
AMAT Reasoning ..... **Error! Bookmark not defined.**
Counter Interpretation ..... **Error! Bookmark not defined.**
Benchmark Pitfall 10
Validation and Reproducibility 10
Conclusion 11
Measurement Limits 11
AI Assistance 11
References 11

Introduction

This project explores how cache residency, access pattern, page locality, and concurrency determine memory performance. A custom benchmark measures serialized read latency, useful bandwidth, and read latency under competing traffic. Elementwise multiplication and AXPY connect these measurements to kernel runtime and reported cache/TLB events. The results show a large penalty for irregular access and diminishing bandwidth gains as more background threads compete for memory.

System Configuration

Item	Configuration and evidence
Processor	AMD Ryzen 5 PRO 7530U with Radeon Graphics; 6 cores and 12 logical processors.
Cache hierarchy	32 KiB L1 data and 512 KiB L2 per core; 16 MiB shared L3. These capacities match the annotations in Figure 1. Aggregate L2 is 3 MiB, not 3 MiB per core.
Host and guest	Windows 11 x64 host. Repeat snapshot: Ubuntu 22.04.3 LTS, WSL2 kernel 6.18.54.1-microsoft-standard-WSL2, x86_64; Python 3.10.12.
Build configuration	Makefile: g++, -O3 -std=c++17 -pthread -fno-tree-vectorize with warnings. Repeat snapshot records GCC 11.4.0 and GNU Make 4.3; original build overrides are not established.
Affinity and concurrency	Probe vCPU 0; workers 2,4,6,8,10,1,3,5,7,9,11. Repeat guest reports six cores, two threads/core, SMT enabled and one NUMA node. Adjacent CPU IDs are siblings.
Power memory and placement	Host RAM channels, power mode, AC state and VM limits are unverified. Guest frequency/governor readings are unavailable. Repeat THP policy is madvise; explicit huge-page pool is zero.

Run Procedure

The original full-profile summary contains 229 configurations with five successful trials each (1145 samples), targeting 250 ms on probe CPU 0. A separate loaded-only repeat targets 2000 ms, with 36 configurations and five successful trials each (180 samples). Figure 4 and all loaded-latency findings use this repeat; the other experiments retain their original results. The two intervals are not pooled. Setup, first touch and a warm pass precede timing; trial order uses seed 7530. The new plan, commands and raw records confirm all 180 planned attempts succeeded.

Elapsed time uses CLOCK_MONOTONIC_RAW. Latency is ns per dependent load, useful bandwidth is counted bytes divided by elapsed ns, and kernel time is ns per update. Statistics use baseline_table.csv and summary(1).csv for the original experiments, and summary(2).csv and knee_candidates(1).csv for the loaded repeat. Calculations use full CSV precision; displayed statistics round to four decimal places. No values are estimated from plot coordinates. Error bars are sample SD, not confidence intervals.

Code Structure and Execution Flow

The benchmark separates measurement, experiment automation and analysis. Direct perf_event_open calls provide the counter interface when the guest exposes the requested events.

Component	Responsibility
src/membench.cpp	Implements pointer chasing, scalar streams, multiply/AXPY, loaded latency, affinity, memory mapping, page-backing inspection and per-thread counters.
Makefile and setup_wsl.sh	Build the C++17 executable, prepare Python dependencies and run implementation checks.

Component	Responsibility
scripts/run.py	Constructs sweeps, selects CPU order, randomizes trials and retains commands, raw measurements, source hashes and system snapshots.
scripts/analyze.py	Normalizes valid events, summarizes repetitions, identifies 90-percent-of-peak candidates and generates the figures.
host_configuration.ps1	Records Windows CPU/RAM, OS, power plan and WSL settings without changing them.
tests/test_suite.py	Checks cycle coverage, byte accounting, kernel output, required configurations, concurrent workers and summary behavior.

Hypothesis

It is expected that serialized latency will rise as the footprint moves beyond each cache level. Sequential access is expected to sustain greater useful bandwidth than shuffled access as adjacent data can be reused and its movement predicted. Additionally larger useful accesses are expected to waste less of each fetched line. Increasing concurrency was expected to improve throughput initially, then raise latency with little additional bandwidth. Spreading accesses over more pages was expected to increase translation pressure, with huge pages helping where translation was a substantial part of the cost.

Experiments and Results

Zero Queue Latency and Working Set

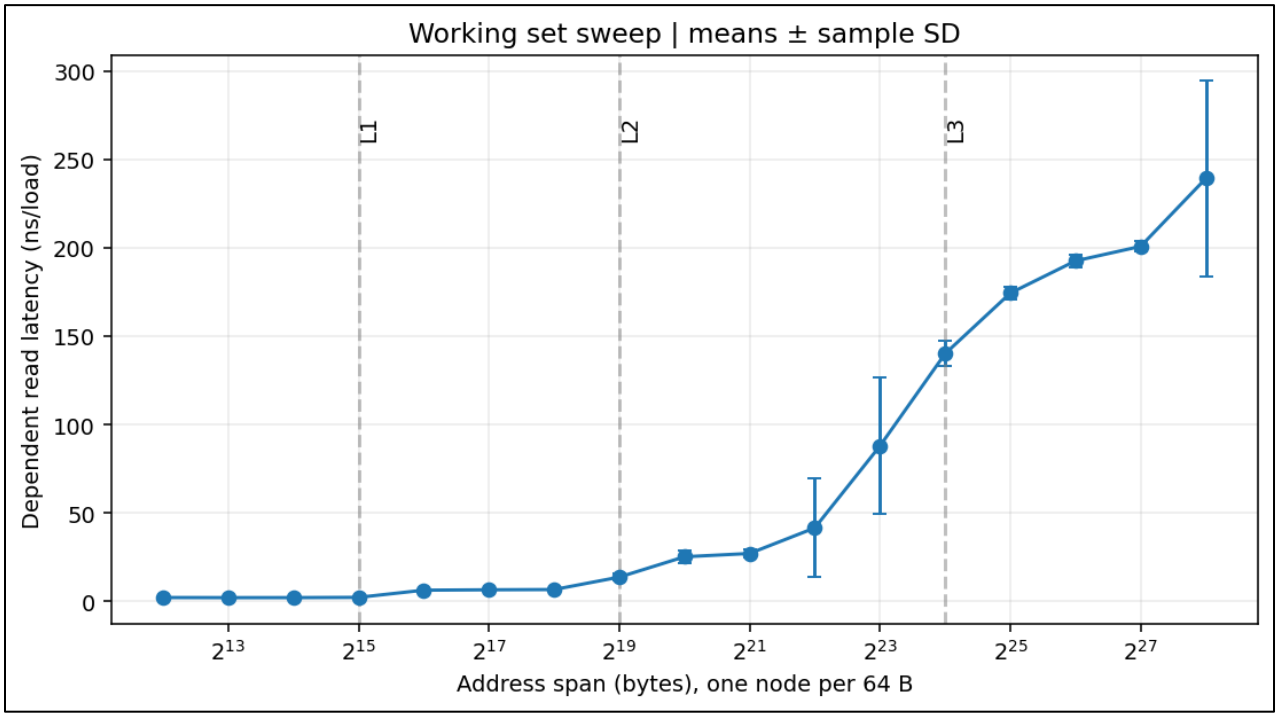


Figure 1. Dependent read latency across working sets. Dashed lines denote L1D, L2 and L3 capacities.

The random pointer chase follows one dependency chain, leaving one benchmark demand load outstanding at a time. This provides a low-intensity baseline, although operating-system traffic, host scheduling, address translation and loop overhead remain.

Region or candidate	Footprint	Latency ns/load mean ± SD	Measured cycles/load mean ± SD
L1D interior	16 KiB	2.0828 ± 0.0080	4.0320 ± 0.0064
L2 interior	256 KiB	7.2201 ± 0.4481	13.8772 ± 0.8143
LLC candidate	8 MiB	86.7116 ± 34.7289	167.3301 ± 67.6408
DRAM candidate	64 MiB	192.3727 ± 4.9214	371.5796 ± 8.3840

The table reports the separate baseline trials in baseline_table.csv, with n = 5 per row. Latency and measured core cycles are shown as mean ± sample SD. The cycle values come from the counter summaries, instead of runtime*nominal 2.00 GHz clock. Across the four candidates, mean counted cycles per elapsed ns range from 1.9224 to 1.9359.

The working-set sweep is a separate set of trials: its 256 KiB mean is 6.6495 ns, versus 7.2201 ns in the baseline. The sweep reaches 25.1890 ns at 1 MiB, 27.0537 ns at 2 MiB, and 239.1866 ± 55.2775 ns at 256 MiB. The LLC baseline has a 40.0510% coefficient of variation and lies on a rising region rather than a clean hit plateau. It cannot be treated as an isolated L3 hit time; translation and effective residency also contribute.

Pattern Granularity and Read Write Mix

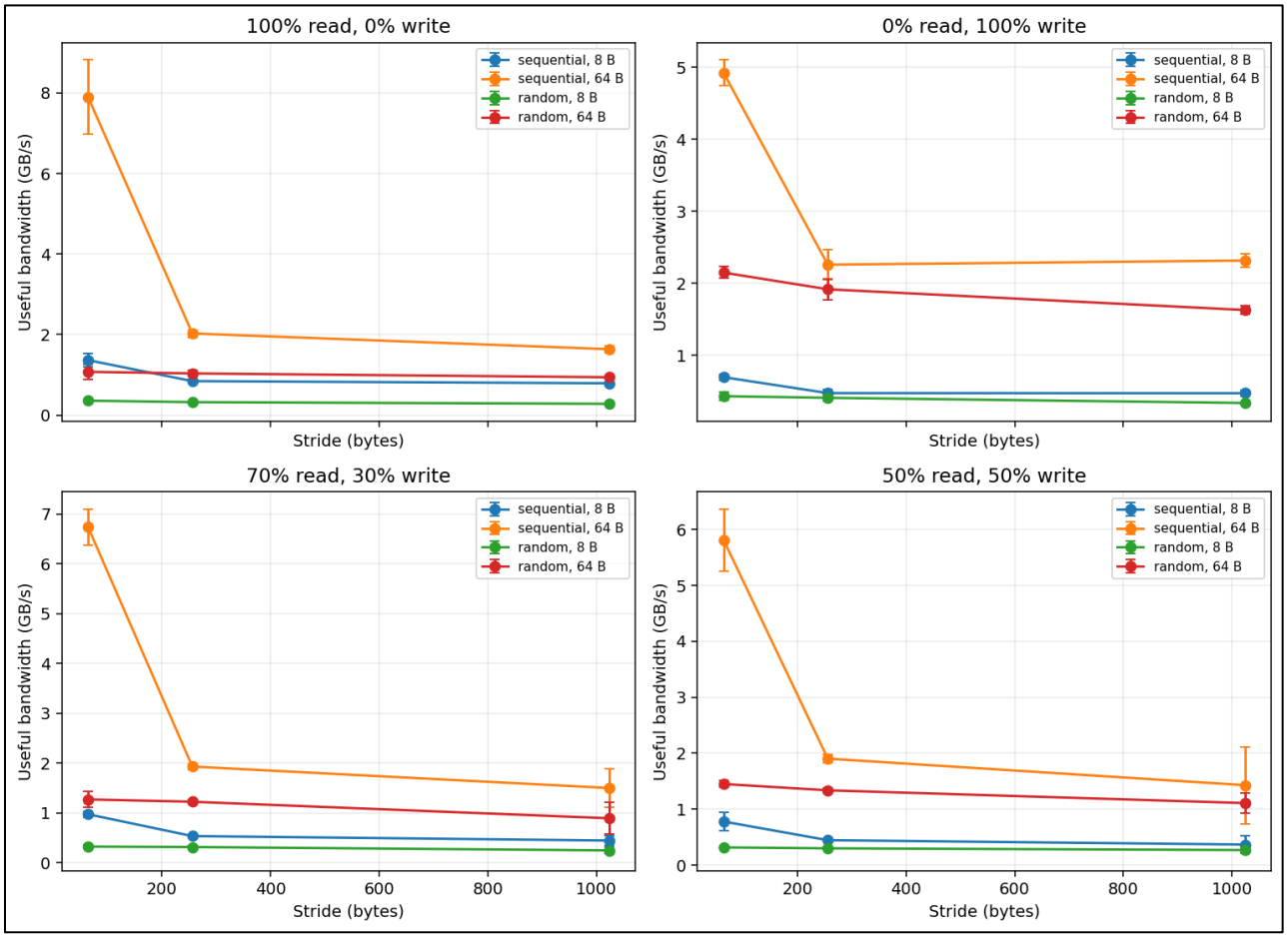


Figure 2. Useful bandwidth for both access patterns, access sizes, strides and read/write mixes. Error bars show sample SD.

At 64-byte stride and 64-byte granularity, sequential useful bandwidth is 7.8990 ± 0.9247 GB/s for reads, 4.9241 ± 0.1803 GB/s for writes, 6.7399 ± 0.3607 GB/s for 70/30 traffic and 5.8156 ± 0.5518 GB/s for 50/50 traffic. These are scalar-generator rates with explicit payload counted in the numerator; index and other physical traffic are excluded.

Read-only random 64-byte access reaches 1.0738 ± 0.1850 GB/s, making the sequential/random ratio 7.3560x. Sequential 8-byte access reaches 1.3642 ± 0.1711 GB/s. Raising granularity to 64 bytes increases useful bandwidth by 5.7904x because more of each fetched line contributes useful payload. This ratio describes useful throughput, not dependent-load latency.

Sequential 64-byte read bandwidth falls to 2.0266 ± 0.0946 GB/s at 256-byte stride and 1.6359 ± 0.0834 GB/s at 1024-byte stride. These are reductions of 74.3439% and 79.2899% from contiguous access. Mix ordering depends on stride: sequential writes at 256 bytes reach 2.2598 GB/s, exceeding reads at 2.0266 GB/s. Store buffering, ownership traffic and generator costs limit interpretation of the mix ordering.

Access Pattern and Dependent Latency

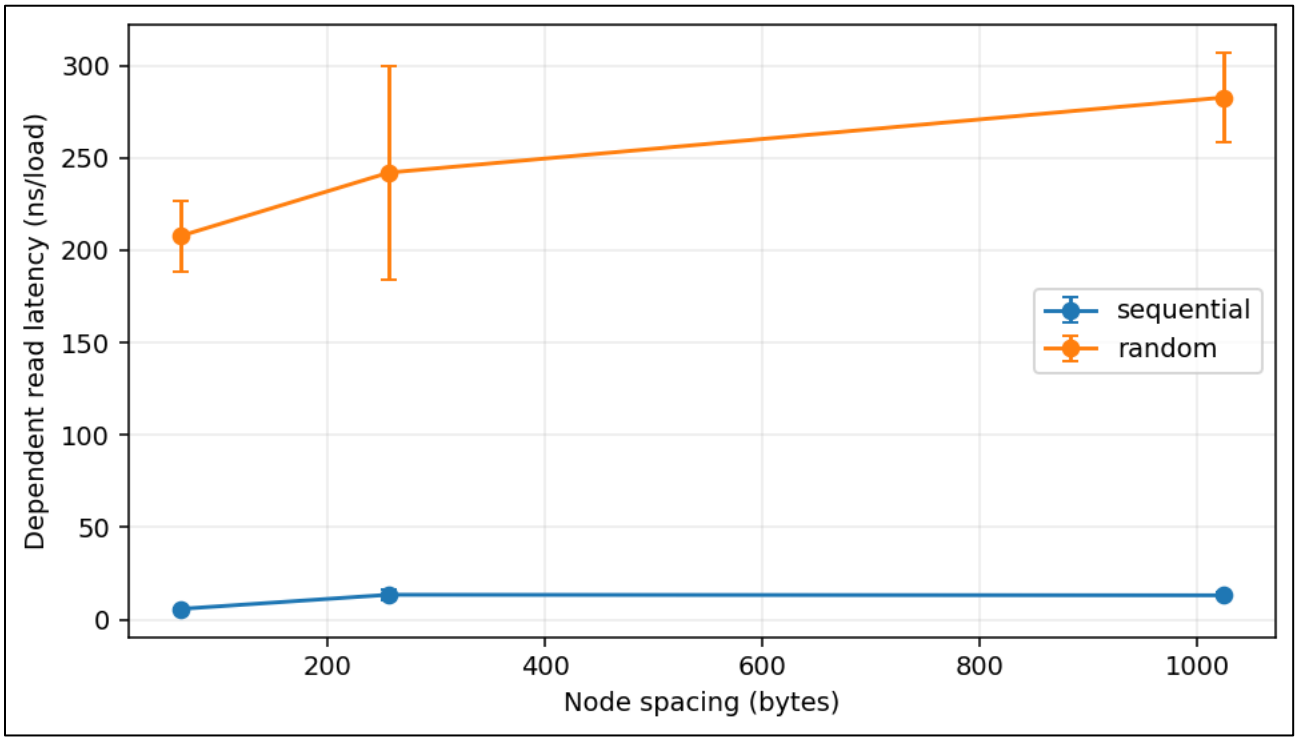


Figure 3. Sequential and random dependent-read latency at three node spacings. The probe loads one 8-byte pointer per node.

Spacing B	Sequential ns/load mean ± SD	Random ns/load mean ± SD	Random/seq
64	5.8039 ± 0.1339	207.7097 ± 18.9530	35.7878
256	13.4971 ± 2.7260	242.0897 ± 57.7839	17.9364
1024	13.1879 ± 1.4463	282.7784 ± 24.3340	21.4423

The table gives the CSV means and SDs for each pattern/spacing pair; n = 5 in every case. Sequential and random access have the same visited-line count at a given configuration, but different address order.

A dependency chain limits demand-load overlap, but predictable successive addresses can still be prefetched. The sequential result therefore reflects the complete access. The random chain reduces this predictability and is more sensitive to cache misses and address translation.

The sweep holds visited line count constant while increasing virtual span. The random/sequential latency ratios are 35.7878x at 64 bytes and 21.4423x at 1024 bytes. The larger span increases translation pressure as well as changing locality. Random 256-byte latency has an SD of 57.7839 ns, so neighboring random means are less clearly separated than the broad sequential/random contrast.

Figure 2 measures throughput as opposed to serialized latency. The read/write latency comparison is provided by the loaded-probe experiment in Figure 4: a read-only dependent probe runs beside load workers using each required mix. In particular, the 100-percent-write series reports read latency under write traffic. It does not measure the service time of an individual store.

Throughput Latency Knee

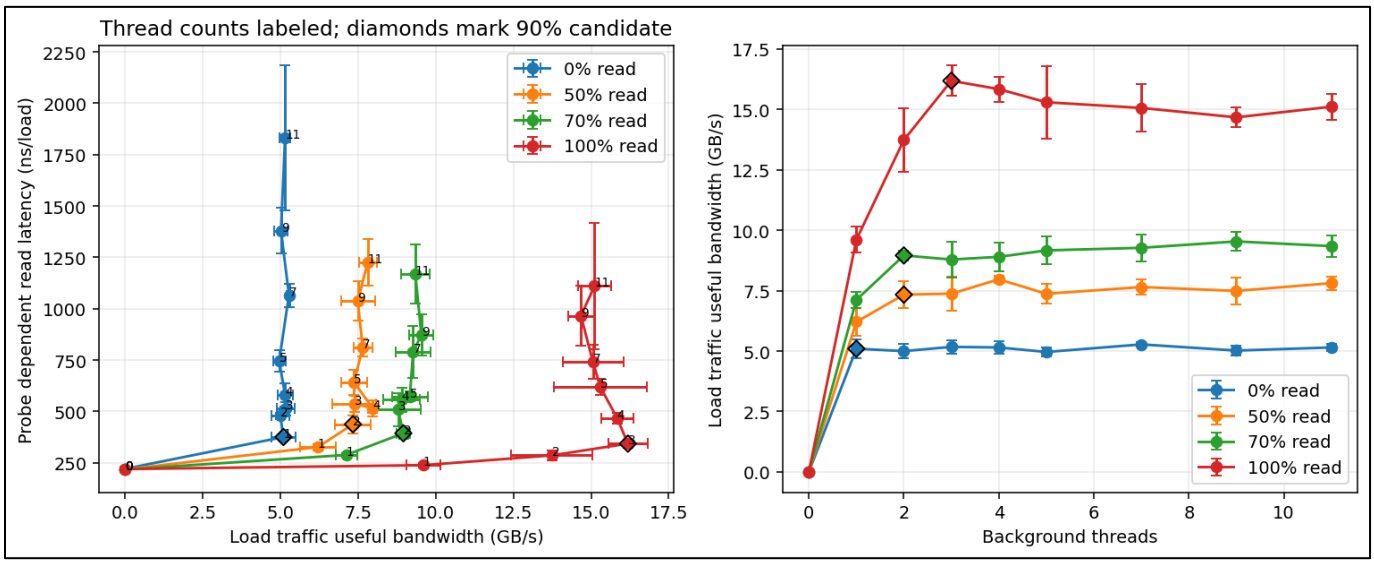


Figure 4. Loaded-only repeat, 2000 ms and five trials per point. Diamonds mark the first worker count at 90 percent of observed peak.

Read/write	Peak GB/s	Candidate workers	Candidate GB/s	Probe ns/load mean ± SD
100/0	16.1852	3	16.1852	343.5156 ± 19.4947
70/30	9.5378	2	8.9580	394.0128 ± 26.9543
50/50	7.9692	2	7.3362	437.9810 ± 44.0119
0/100	5.2767	1	5.0991	375.8315 ± 13.4924

For read-only load, the no-load probe is 219.7358 ± 13.5121 ns. Bandwidth rises from 9.6047 to 13.7338 to 16.1852 GB/s with one, two and three workers. From three to five workers, bandwidth falls to 15.2968 GB/s while latency rises from 343.5156 to 619.3935 ns. Three workers remain the read-only operational knee under the longer interval.

Mixed/write candidates remain at two, two and one workers for 70/30, 50/50 and write-only traffic. At eleven workers, 100/0, 70/30, 50/50 and 0/100 probe latencies are 1110.7342, 1171.1809, 1225.0832 and 1831.7258 ns. The guest topology reports adjacent SMT pairs: 0/1, 2/3, 4/5, 6/7, 8/9 and 10/11. Workers use 2,4,6,8,10 before siblings, with the probe on 0; seven workers include its sibling 1. The candidate is the first worker count reaching 90% of that mix's observed peak, not proof of DRAM-bus saturation.

Concurrency and Queueing

Little's Law gives N = lambda x W for the same steady-state request population. With true 64-byte transaction bandwidth, lambda = B/64. At the read-only knee, the illustrative proxy is 16.1852 x 343.5156 / 64 = 86.8729, calculated using unrounded CSV inputs. Useful stream bandwidth and separate probe latency represent different traffic populations; this is not measured queue depth or outstanding-request count. The observed trend is increasing response time with diminishing throughput gains.

Kernel Runtime and Cache Miss Impact

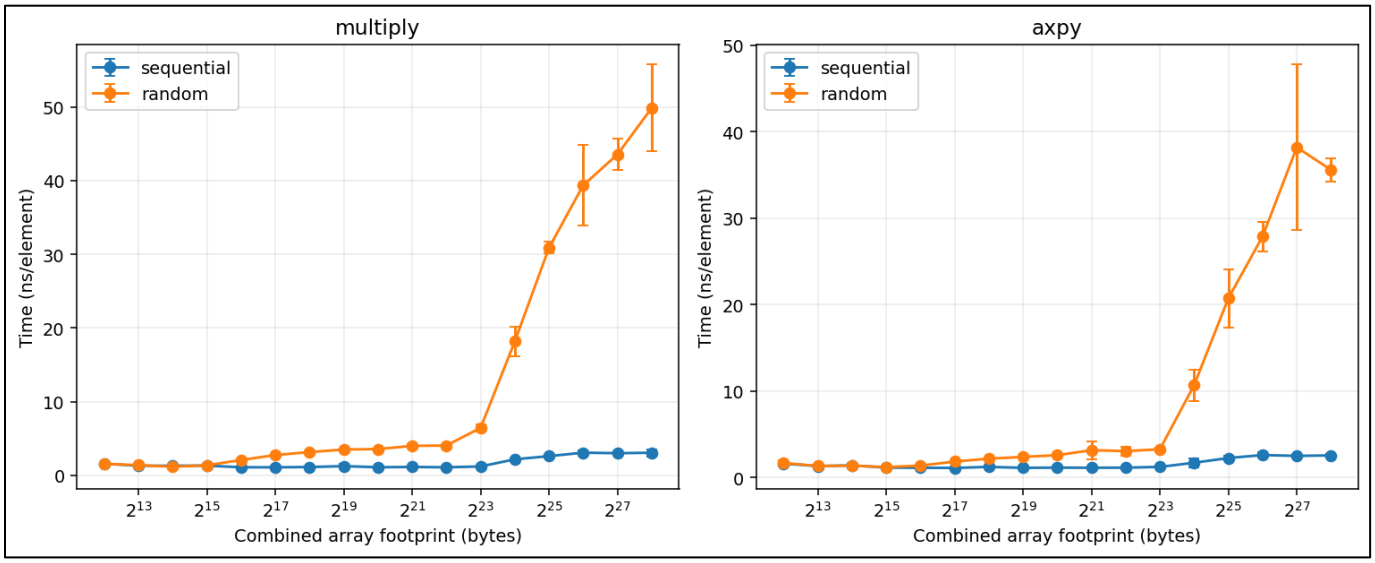


Figure 5. Time per element for scalar multiplication and AXPY across combined array footprints.

At 256 MiB, multiplication takes 3.0785 ± 0.4996 ns per sequential update and 49.8425 ± 5.9070 ns randomly, a 16.1906x penalty. AXPY takes 2.5432 ± 0.1359 ns sequentially and 35.5758 ± 1.3404 ns randomly, a 13.9884x penalty. Both random curves rise beyond the smaller cached footprints, while sequential execution changes much less.

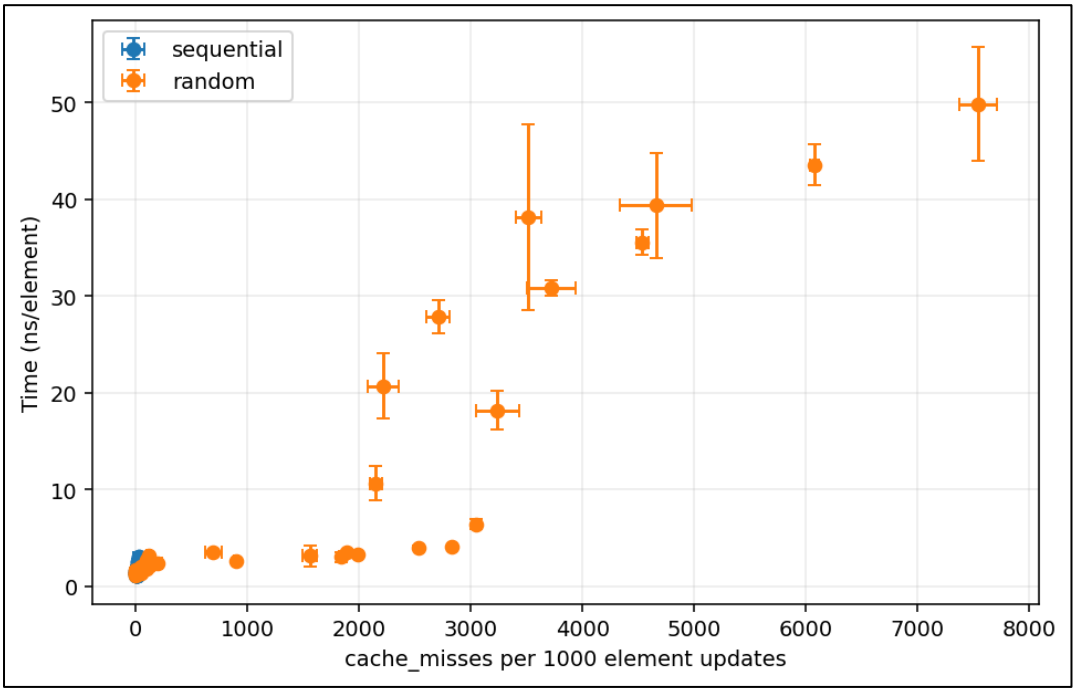


Figure 6. Kernel time versus reported generic cache-miss events per 1000 element updates. Both kernels are included.

At 256 MiB, reported generic cache-miss events per 1000 updates rise from 27.7093 to 7536.8184 for multiplication and from 16.9881 to 4536.8575 for AXPY when order becomes random. This associates poor locality with longer runtime. Events per update are not a miss percentage or a verified physical-traffic count. The generic event mapping and pooled kernels prevent treating the plotted slope as a single cache-miss penalty.

Page Locality and Huge Pages

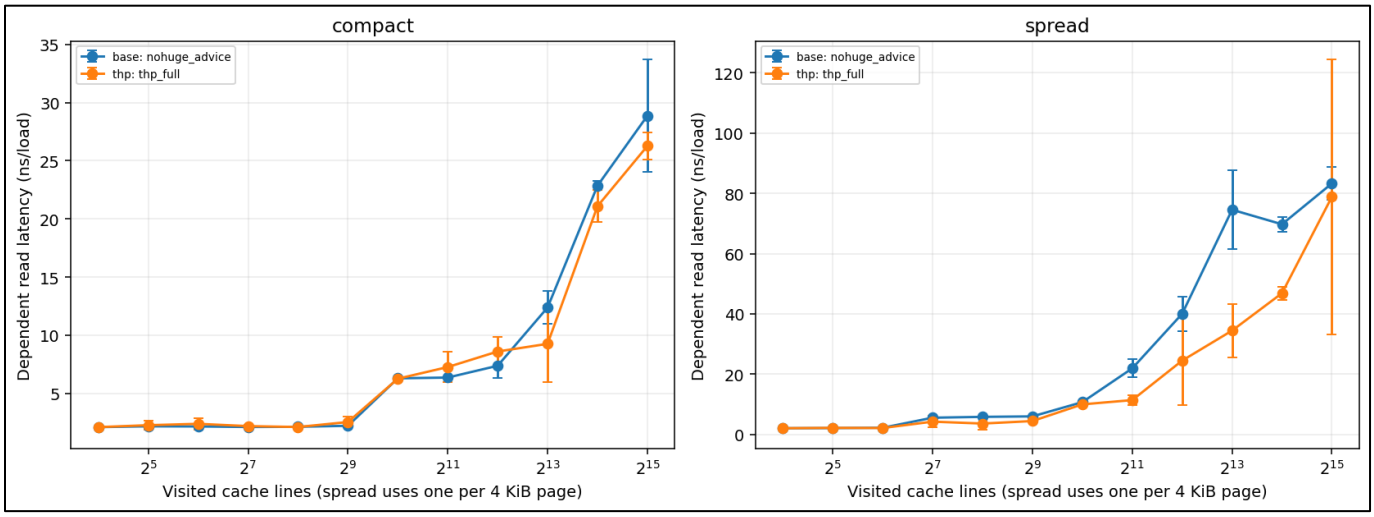


Figure 7. Equal visited-line counts under compact and spread layouts. The THP series is labeled thp_full by the analyzer.

At 8192 nodes, compact/spread layouts both visit 512 KiB of cache lines over 512 KiB/32 MiB nominal spans. Base-page latency is 12.3729 ± 1.4256 ns compactly and 74.5496 ± 13.0979 ns spread out. Spread THP latency is 34.4577 ± 8.9091 ns, a 2.1635x improvement. At 16384 spread nodes, base/THP means are 69.7292/46.8626 ns. At 32768 nodes they are 83.2694 ± 5.3913/78.8353 ± 45.5320 ns; the large THP SD limits the largest-case benefit claim.

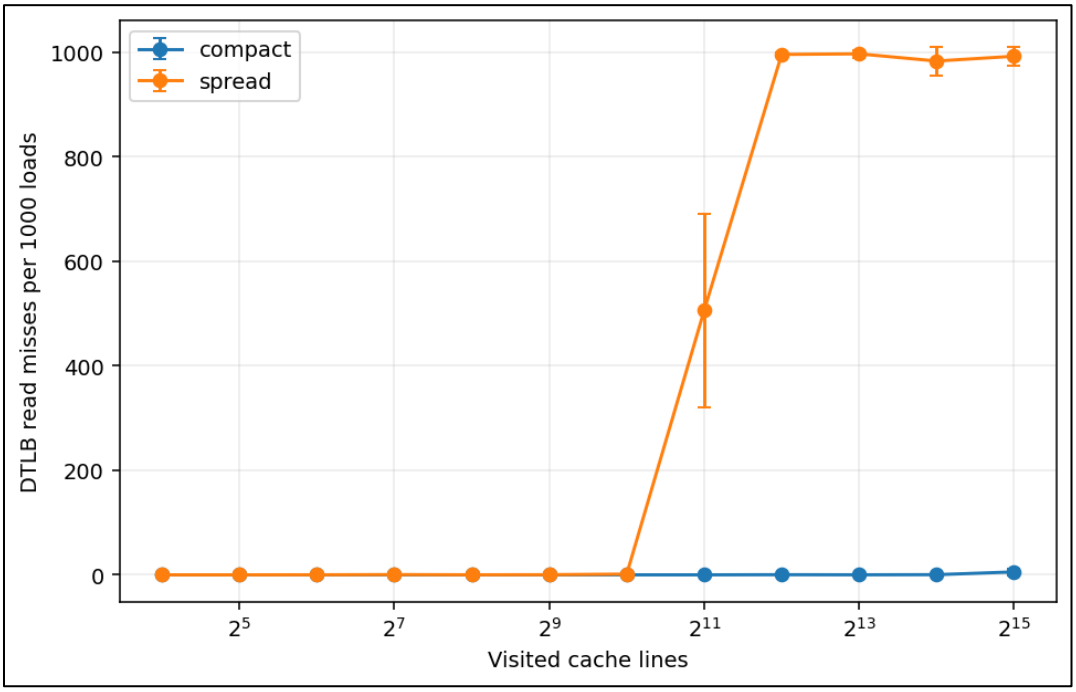


Figure 8. Reported DTLB read misses per 1000 loads for the base-page compact and spread tests.

Base-page spread DTLB read-miss events per 1000 loads are 1.4352 at 1024 nodes, 505.8335 at 2048 and 995.5336 at 4096. Compact 4096-node intensity is 0.2932. This supports a translation-pressure explanation without identifying exact TLB capacity. All 24 THP groups report thp_full.

TLB Misses and AMAT Interpretation

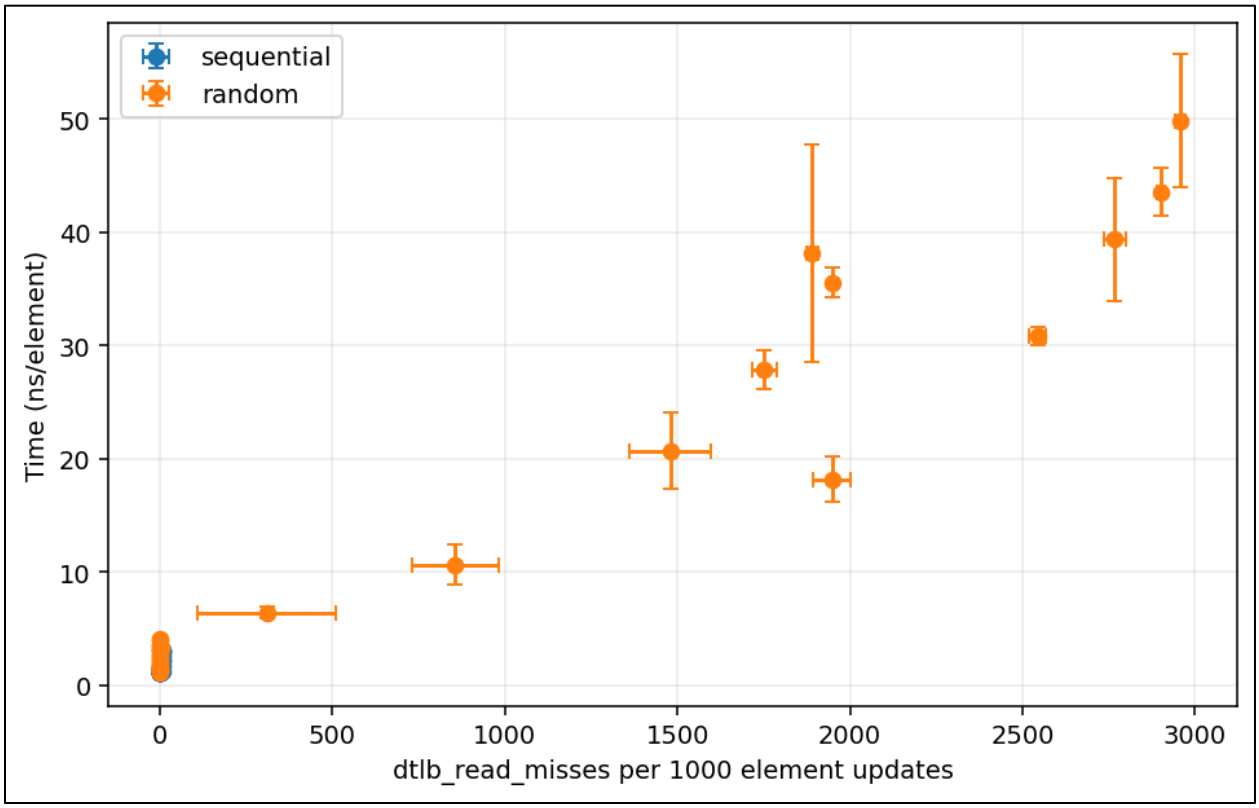


Figure 9. Kernel time versus reported DTLB read-miss events per 1000 element updates. These are kernel updates, not the one-load probe denominator.

At 256 MiB, reported DTLB read-miss events per 1000 updates are 6.8424 sequentially and 2958.3936 randomly for multiplication; AXPY gives 5.3160 and 1948.4866. Matched 8192-node spread tests also show a THP reduction from 996.6110 to 302.2106, or 69.6762%. This is counter evidence for the observed THP timing improvement, while cache/translation effects still overlap.

A hierarchical average-memory-access-time model can be written as:

AMAT = tL1 + mL1 x (tL2 + mL2 x (tLLC + mLLC x tDRAM))

Each m is a conditional miss probability for accesses reaching that level, and the t terms are incremental service costs. Reduced residency makes slower service contribute more often. Figures 1, 5 and 6 are consistent with this reasoning, while Figures 7-9 show that address translation also changes the observed cost.

The generic miss events do not supply every conditional probability in the formula, and Figure 1 gives end-to-end load times rather than isolated incremental penalties. Substituting its four latency readings directly into all AMAT terms would double-count costs. A numerical AMAT prediction is therefore not established here. The measured result is that runtime rises with irregular access, larger footprints and greater reported cache/TLB event intensity.

Multiplication at 256 MiB retires 12.0003 instructions/update sequentially and 12.0025 randomly, while measured cycles/update rise from 5.8059 to 89.5126. Baseline instructions/load change from 3.0229 in L1 to 3.0419 in the DRAM candidate, while CPI rises from 1.3338 to 122.1523. More costly memory service, rather than a large instruction-count increase, is consistent with these data. Five counter samples support each reported mean. Raw event status and enabled/running times are still needed to audit scaling and mapping. [1]

Benchmark Pitfall

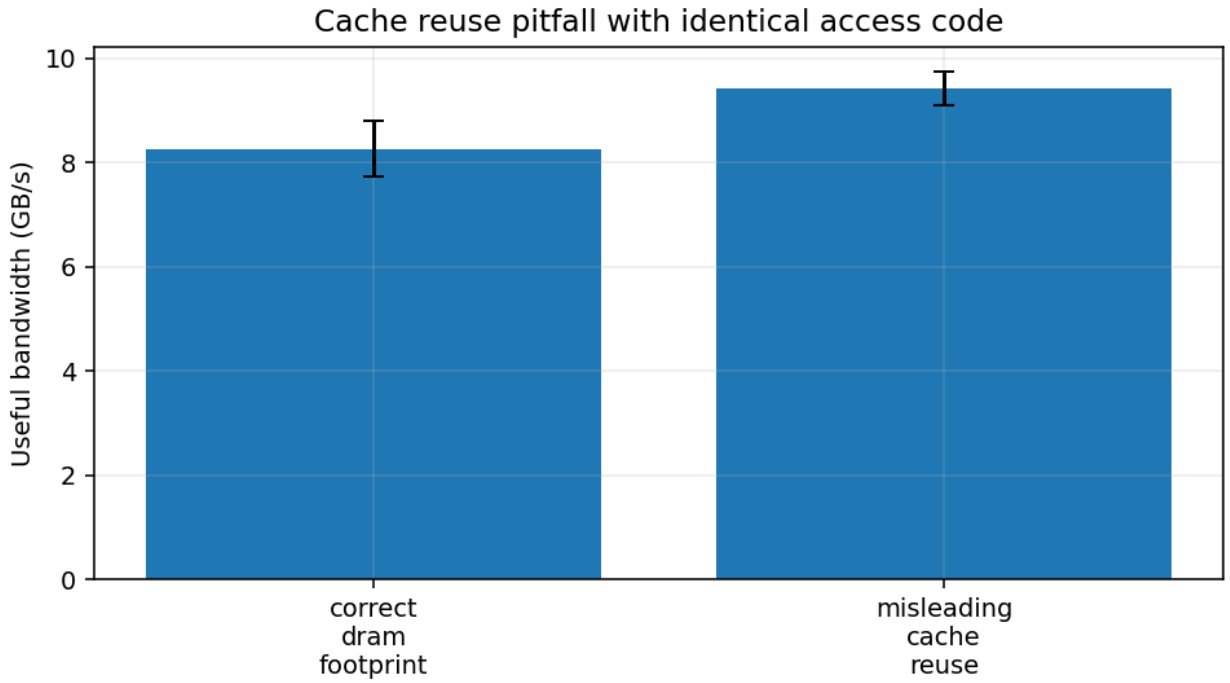


Figure 10. The same read-only stream kernel with a DRAM-candidate footprint and a warmed cache-resident footprint.

The 16 KiB cache-reuse case reaches 9.4163 ± 0.3240 GB/s, compared with 8.2602 ± 0.5367 GB/s for the 64 MiB DRAM candidate, with n = 5 each. The mean ratio is 1.1400x. Calling the small-case result DRAM bandwidth would overstate the larger-case mean by 13.9970%. Footprints and statistics are confirmed directly by the summary.

The mean difference is modest. The scalar loop pays indexing, checksum and access-selection overhead, and fences once per array pass, so small and large footprints amortize overhead differently. The result demonstrates a labeling error: warmed cache service cannot establish DRAM bandwidth. Even the larger result remains useful bandwidth rather than a measured bus rate. No significance test or confidence interval for the mean ratio is available from these summaries.

Validation and Reproducibility

The implementation checks chase coverage, stream stores, numerical kernel output, byte accounting, sweep completeness, loaded workers and missing counters. Development tests passed. The original processed tables contain five retained trials in each of 229 groups; baseline and knee tables agree with their summary. For the repeat, all 900 numeric summary fields were independently reproduced from 180 raw JSONL trials. Every probe records CPU 0 at start and end; all 36 planned groups have five successful trials and no failed or skipped attempts.

The summary contains 17 working-set, four baseline, 48 pattern/mix, six pattern-latency, 48 page-locality, 68 kernel, 36 loaded and two pitfall groups. All expected non-HugeTLB configurations are represented. The 24 planned explicit-HugeTLB configurations have no successful summary rows; their attempt statuses require audit.json or raw.jsonl. The source, commands, raw samples, processed tables and plotting scripts remain part of the submission.

Workers finish their array pass after the probe stops; aggregate bandwidth sums their individual active-time rates. The original 250 ms run had tails above 5% in 24 of 32 nonzero-load groups, with a maximum of 39.0197%. At 2000 ms, no group exceeds 5%; the maximum is 4.7190% at eleven write-only workers. The read-only peak changes from 17.7752 to 16.1852 GB/s across runs, while candidate counts remain unchanged. Duration and collection time both changed, so the peak difference cannot be attributed solely to timing tails. Loaded mode intentionally collects no PMU/fault metrics; missing events are not failures or evidence of zero faults.

Conclusion

The experiments show that locality and competing traffic strongly affect useful memory performance. Separate baseline trials give 2.0828 ns/load in the L1 candidate and 192.3727 ns/load in the DRAM candidate. The LLC candidate remains variable rather than an isolated hit plateau. Contiguous 64-byte sequential reads deliver 7.3560x the random useful bandwidth. At 256 MiB, random order costs 16.1906x the multiplication runtime and 13.9884x the AXPY runtime. Reported cache and TLB event intensities rise alongside these costs.

Read-only useful load bandwidth peaks at 16.1852 GB/s with three workers in the 2000 ms repeat. Further workers raise probe latency without increasing peak throughput; mixed/write traffic reach their 90-percent candidates earlier. The repeat reduces all timing tails below 5%, supporting the operational knee counts. At 8192 spread nodes, THP gives a 2.1635x timing improvement and a 69.6762% reduction in reported DTLB intensity. The practical result is to preserve line/page locality and avoid adding concurrency after useful throughput stops improving.

Measurement Limits

Statistics use CSV values, with four decimal places displayed. The loaded repeat includes raw records, audit and guest snapshots; original full-run event scaling, failures and explicit HugeTLB outcomes still need their original raw records. Windows power mode, AC state, RAM channels, thermals and VM limits are not established. Scheduling, SMT and frequency changes affect results; guest affinity does not reserve physical cores. At eleven write-only workers, latency SD is 352.0711 ns despite tails below 5%. Warm reuse, prefetching, index traffic, scalar execution and temporal stores prevent these rates from being universal hardware limits.

All THP groups report full backing. At 8192 spread nodes, the minimum recorded anonymous huge backing is 32768 KiB; corresponding base-page groups record zero. The CSVs now provide matched THP/base counter evidence as well as timing. Explicit HugeTLB outcomes and raw mapping/event checks remain unverified. Generic cache/TLB counts are event intensities, not unconditional miss probabilities, and cannot supply all conditional inputs for a numerical AMAT calculation. [2]

AI Assistance

ChatGPT generated the benchmark and supplemental scripts, and assisted with interpretation and report wording. Measurements come from the laptop's WSL runs. Numerical statements use the supplied CSVs, with display rounding only. The loaded-repeat summary was cross-checked against its raw trials; guest settings are identified from that repeat's metadata.

References

[1] Linux man-pages project. perf_event_open(2). Counter setup, event groups and scaling. https://man7.org/linux/man-pages/man2/perf_event_open.2.html. Accessed October 23, 2026.

[2] Linux kernel documentation. Transparent Hugepage Support. https://docs.kernel.org/admin-guide/mm/transhuge.html. Accessed October 23, 2026.

[3] Microsoft Learn. Advanced settings configuration in WSL. https://learn.microsoft.com/en-us/windows/wsl/wsl-config. Accessed October 23, 2026.